
Layered Formal Verification of the Optimistic Democracy Protocol

José María Lago Albert Castellana Edgars Nemše

GenLayerLabs Research
jm@genlayerlabs.com albert@genlayerlabs.com edgars@genlayerlabs.com

August 2026

Abstract

Statistical treatments of panel decisions and idealized appeal ladders give guarantees under explicit probability models; those results do not verify the deployed selection law or state machine. This paper addresses the implementation side of that separation. We present a layered verification architecture for a deployed, actively-developed consensus protocol: a canonical state graph as the single machine-readable source of truth; automated structural proofs over the graph; a bidirectional coverage mapping between graph arcs and Solidity code paths, mechanically anchored to a pinned revision of the contracts; and TLA+ specifications—one concrete, one counter-abstracted—whose transition topology is automatically checked against the canonical graph. The architecture is designed for a moving target: every layer re-verifies cheaply as the implementation evolves.

We report a case study in which re-verification against four months of protocol development (253 commits) surfaced a divergence between two code paths handling the same protocol event—producing materially different outcomes depending on transaction ordering—and, via model checking, three protocol invariants that a single hotfix silently falsified: commit–reveal integrity, the leader-reveals-first ordering, and quorum–outcome consistency. All findings were confirmed against the implementation and reported upstream. We argue that for protocols under continuous development, the primary value of a formal specification is not the initial proof but its use as a semantic regression harness.

Keywords: TLA+; model checking; formal verification; consensus protocols; semantic regression; smart contracts

1 Introduction

Companion statistical work establishes mathematical guarantees for narrower declared models: exact panel–population laws, sufficient bounds, operational certificates, and adversarial profiles for threshold voting, together with an analysis of an idealized appeal ladder under additional committee and cross-round assumptions. Neither analysis proves that the deployed stake-weighted sampler or Solidity state machine realizes those models.

This is not a hypothetical concern. The implementation we verify is a production codebase under continuous development: between our first verification baseline and the present revision, the consensus contracts absorbed 253 commits, including hotfixes that changed state transitions. A proof about yesterday’s state machine is silent about today’s.

1.1 Contributions

1. **A layered verification architecture** (Section 3) connecting four artifacts—a canonical state graph, automated structural proofs, a Solidity coverage map with mechanical anchor checking, and TLA+ specifications—such that each layer is checked against its neighbors automatically. The design goal is cheap re-verification against a moving implementation, not a one-time certificate.
2. **TLA+ specifications of Optimistic Democracy** (Section 4): a concrete specification of the full transaction lifecycle (14 on-chain statuses, appeals, idleness recovery, tribunal), and a counter-abstracted variant that trades per-validator identity for scalability, with a quantified account of the state-space engineering involved.

3. **A case study in semantic regression** (Section 5): re-verification against four months of protocol development surfaced (a) a race between two entry points handling the same hanging-appeal event with divergent outcomes, and (b) three previously-true protocol invariants falsified by a single hotfix—each confirmed in the implementation and reported upstream.
4. **A verification-status discipline** (Section 6): every claim in the verification pipeline is labeled exhaustive, partial, or pending, with the boundary mechanically visible. We report what did not finish alongside what did.

1.2 Scope

We verify the *consensus transaction lifecycle* and its adjacent subsystems (tribunal appeals, staking lifecycle, transaction queues) as implemented in the GenLayer consensus contracts. Probabilistic properties (convergence rates, adversarial margins) are the subject of the companion statistical treatment and are not re-proven here; the incentive properties of appeals are treated in companion work on appeal games. The governance subsystem has a parallel specification track in the same architecture; its verification is ongoing and explicitly out of scope for the claims of this paper.

2 The Protocol Under Verification

We summarize the implemented protocol to the depth verification requires; companion papers develop the underlying statistical and economic theory.

2.1 Transaction lifecycle

A transaction proceeds through a lifecycle of 14 on-chain statuses (plus one virtual, computed status): from `Pending`, through activation, leader proposal (`Proposing`), a commit–reveal vote among a sampled validator committee (`Committing`, `LeaderRevealing`, `Revealing`), to a decided status (`Accepted`, `Undetermined`, `LeaderTimeout`, `ValidatorsTimeout`), an appeal window, and finalization (`Finalized`) or cancellation (`Canceled`). Appeal rounds introduce two further statuses (`AppealCommitting`, `AppealRevealing`). The virtual status `ReadyToFinalize` is computed by a view function when the appeal window has expired and is never stored.

2.2 Committees, votes, appeals

Round-zero committees have 5 validators. The pinned `FeeManager`’s nominal size entries are

$$5, 7, 11, 13, 23, 25, \dots, 767, 769, 1535, 1537.$$

This array is not a path-independent appeal schedule. The contract checks appeal feasibility from the current round and the available unconsumed validator count, while the companion specification applies a minus-two size adjustment after consecutive unsuccessful appeals. In the documented $N = 1543$ capacity scenario, eight appeal juries through size 769 can lead to a 1535-member normal successor. Successor rounds reset ballots, but eligible validator identities may be reused. The pinned voting contract computes quorum from the realized committee length, so that successor’s positive threshold is 768; no fixed threshold 772 is implemented there.

Decisions distinguish agree, timeout, disagree, deterministic violation, and no-majority categories. The projection “Accepted iff agree has a strict majority” matches the inclusive binary threshold form at $\tau = 1/2$ studied in the companion statistical treatment on these odd committee sizes; the richer status machine, appeal-success comparison, path-dependent sizing, and successor construction do not. Validator appeals may be submitted from `Accepted`, `ValidatorsTimeout`, or—in the current revision—`Revealing`; leader appeals from `Undetermined` and `LeaderTimeout` re-enter the proposal flow. The pre-appeal status is recorded (`previousStatus`) and is restored by several recovery paths, a mechanism that features centrally in Section 5.

2.3 The moving target

Three properties of this codebase shape the verification problem. First, it is *phase-structured*: status transitions are scattered across phase contracts (activation, proposal, committing, revealing, finalization, idleness) plus helper libraries, so no single contract exhibits the state machine. Second, it is *actively developed*: transitions move between files, line numbers drift, and hotfixes change semantics. Third, it has *multiple entry points for the same protocol*

event: for instance, a hanging transaction can be detected inline during a vote submission or by a dedicated idleness sweep—and these paths are maintained separately.

3 The Verification Architecture

The architecture is a chain of artifacts, each mechanically checked against its neighbors:

Layer	Artifact	Checked against
L0	Solidity contracts (pinned revision)	— (ground truth)
L1	Canonical state graphs (Python)	L0, via L2
L2	Transition map + anchor check	L0 (mechanical), L1 (bijection)
L3	Structural proofs	L1
L4	TLA+ specifications	L1 (topology check)
L5	Model-checked properties	L4 (TLC)

3.1 L1: canonical graphs as the single source of truth

The consensus lifecycle is encoded as a labeled directed graph in a small Python module: 14 real states (on-chain statuses), 9 *decision nodes* (conceptual branch points that never appear on-chain, e.g. “who is idle?”), 64 labeled transitions. Collapsing decision nodes yields 42 *effective arcs* between real states. Three further graphs cover the tribunal-appeal lifecycle (6 states, 7 arcs), the staking lifecycle (12 states, 15 arcs), and the transaction queue (8 states, 14 arcs); a composite graph is derived programmatically. Everything downstream—documentation diagrams (generated Mermaid), the coverage map, the TLA+ topology check—consumes this module. There is exactly one place where the state machine is defined.

3.2 L2: Solidity coverage with mechanical anchoring

Proof 2 is a bidirectional mapping between effective arcs and Solidity code paths. The transition map contains 83 entries, each recording the arc, the file, the function, the line of the status-changing call, and a category (normal, idleness, appeal, admin, virtual). Two automated checks run over it:

- **Bijection** (completeness and soundness): every effective arc has at least one Solidity entry, and every entry’s arc exists in the graph.
- **Anchoring**: every entry’s (file, function, line) reference is checked against an actual checkout of the contracts—the referenced line must sit inside the named function and adjacent to a status-changing statement. The checker pins the verified baseline revision and warns when the checkout drifts from it.

The anchoring check exists because of a failure mode we encountered: the bijection is a self-consistency check between two artifacts we control, and it stays green when the *implementation* moves. Between February and June the contracts absorbed 253 commits while all bijection checks passed; the map was silently four months stale. Anchoring makes that drift a CI failure rather than an archaeology project (Section 5).

3.3 L3: structural proofs

Graph-theoretic invariants checked without reference to Solidity: reachability of every node from entry points; a path to a terminal state from every non-terminal node (liveness at the graph level); terminal states have no outgoing arcs; decision nodes branch; cycles occur only in expected subgraphs (rotation and idleness loops).

3.4 L4: TLA+ with automatic topology conformance

Section 4 describes the specifications. The binding to L1 is a topology checker that parses the TLA+ next-state actions, extracts the induced status-transition relation, and compares it against the canonical effective arcs. At the current baseline the concrete specification covers 42/42 arcs. Extra arcs in the TLA+ relation (administrative cleanup paths,

modeling self-loops) are reported but tolerated; missing arcs fail the check. This is what makes the graph, and not the TLA+ file, the source of truth: when the protocol changes, the graph is updated from the Solidity evidence, and the topology check tells us exactly which TLA+ actions are out of date.

4 The TLA+ Specifications

4.1 Concrete specification

The concrete specification models a single transaction’s lifecycle with an explicit validator set: per-validator commits and reveals, leader election and rotation, idleness detection and replacement, the appeal ladder with committee escalation, deterministic-violation tribunals running in parallel, and finalization. State comprises the transaction status, round and appeal counters, the committee as a set, per-phase vote bookkeeping, and tribunal state. Fairness assumptions are stated per action; liveness properties are checked under weak fairness of the progress actions.

4.2 Abstraction strategy

Protocol properties depend on *counts* (quorum, majority) rather than *identities* (who voted what). The abstract specification exploits this in two phases:

1. **Functions** → **counters** (done): per-validator vote functions are replaced by counters (`agreeCount`, `disagreeCount`, `timeoutCount`, hash-match counters), reducing the per-round state from $O(5^N)$ to $O(N^k)$.
2. **Sets** → **counters** (pending): committee and participation sets still contribute $O(2^N)$ configurations; replacing them with cardinalities is the known next step, required to reach committee sizes in the hundreds.

The quantitative effect is documented in Section 6; the abstraction boundary is also an honesty boundary—claims about large committees are made only where the abstract spec, not the concrete one, has been checked.

4.3 Property catalog

The properties fall into four groups (the full catalog is in Appendix A):

- **Commit–reveal integrity** (CR): reveals never exceed commits; a transaction in the reveal phase has the leader’s commit in its current round.
- **Safety of outcomes** (S): unique result per transaction; leader-reveals-first ordering in normal rounds; quorum–outcome consistency (an `Accepted` status with a majority-agree result is backed by a quorum of agree votes); committee sizing follows the escalation schedule. The current target property is that successor rounds reset ballots while validator identities may be reused; the pinned TLA+ campaign predates that clarification and must be re-run before S6 is claimed for the current implementation.
- **Liveness** (L): every transaction eventually reaches a terminal status; pending failovers, pauses, and queues drain.
- **Cross-cutting**: type correctness; tribunal isolation (a tribunal never changes the transaction status directly).

Three of these—CR2, S3, S5—carry documented *exemptions* introduced by the case study of the next section. The exemptions are not weakenings of convenience: each marks a state that the current protocol genuinely reaches and the pre-hotfix protocol did not.

5 Case Study: Re-verification Against a Moving Protocol

5.1 Setting

The verification corpus was built against a February revision of the contracts. Four months later, the implementation head was 253 commits ahead: a new fee-locking path, relocated reveal-phase logic, new recovery helpers, and—critically—a hotfix (internally `CON-441`) changing how a hanging appeal round is handled. All bijection and structural

checks were green throughout, because they are internal to the specification corpus. Re-verification proceeded in three mechanical steps: (i) sweep every status-changing call site in the new revision (46 sites) and diff against the transition map; (ii) re-anchor every map entry to the new revision; (iii) re-run the topology check and the model checker against the updated graph.

5.2 Finding 0: a divergent-handler race

Finding 5.1 (Entry-point divergence, reported upstream). The hanging condition for a transaction in `AppealCommitting` is detected by one shared helper but handled by two independently-maintained code paths. The idleness sweep (post-hotfix) slashes non-committed validators and *reverts the transaction to its pre-appeal status* with a reduced appeal window; the inline path in the vote-submission function (pre-hotfix behavior, unchanged) sends the transaction to `Undetermined` with no slashing. Which outcome occurs depends on which call lands first.

The race has material consequences: in one branch the pre-appeal decision survives and non-committers are slashed; in the other the decision is discarded and no one is slashed. Both branches are live at the current implementation head. In the specification, both handlers are modeled as nondeterministic alternatives of the same action—the nondeterminism *is* the race—so every downstream property is checked against both. The finding was confirmed by direct code inspection and filed upstream with permalinks to both paths.

5.3 Findings 1–3: invariants falsified by the hotfix

Modeling the new revert path and re-running TLC produced counterexample traces against three invariants that had held for the pre-hotfix protocol. Each was analyzed against the implementation and confirmed to be a genuinely reachable on-chain state, not a modeling artifact. The mechanism is common to all three: the revert restores the pre-appeal *status* while the transaction’s current round remains the voided appeal round, whose vote data is empty or partial.

Finding 5.2 (Commit–reveal integrity). A transaction in `Revealing` may have zero commits in its current round, or partial commits that do not include the leader’s. *Falsified invariant*: “a transaction in the reveal phase has the leader’s commit in its current round.” *Exposure*: any code path, event consumer, or indexer assuming that a revealing transaction has a proposed receipt and a committed leader in its current round.

Finding 5.3 (Leader-reveals-first ordering). The leader-reveals-first security ordering is enforced by state flow, not by an explicit check, and the revert bypasses the flow: a transaction can sit in `Revealing` with no leader reveal in its current round. *Exposure*: whether reveals against the voided round are accepted (breaking the ordering assumption) or rejected (making the reduced appeal window the only exit) is an implementation decision that should be explicit; today it is an accident of guard placement.

Finding 5.4 (Quorum–outcome consistency). An `Accepted` transaction’s current-round vote counters may be empty or unrelated to its acceptance (the acceptance was earned in the pre-appeal round). *Exposure*: fee distribution and majority/minority reward classification read round vote data; after a voided appeal round, “the majority” is undefined unless the round-type taxonomy classifies voided rounds explicitly. The appellant’s bond and the slashed validators’ penalties likewise need a defined destination.

5.4 Repairing the properties honestly

The three invariants were not deleted; they were weakened by the *minimal exemption that names the new state*. The specification already tracked a flag marking restored-after-unsuccessful-appeal states (used by the pre-existing appeal-failure restore path); the revert path now sets the same flag, and each exemption is keyed on it. The residual properties still check everything else: any nonempty reveal set must contain the leader in normal rounds; quorum consistency holds whenever counters are live; commit–reveal inequalities hold unconditionally. For the leader-first property the exemption is semantically exact rather than a compromise: the voided round is an *appeal* round, and appeal rounds have no leader-first rule by design.

5.5 What the case study shows

Two lessons generalize. First, **self-consistency is not verification**: a specification corpus whose checks never touch the implementation will remain green while the implementation walks away from it; at least one check must anchor to the

code, mechanically, at a pinned revision. Second, **the specification is most valuable after the initial proof**: the initial verification found modeling bugs; the *re*-verification found protocol bugs. A hotfix reviewed and merged by competent engineers falsified three invariants nobody knew the protocol had been relying on, and a model checker found all three in minutes once the change was modeled. For a protocol under continuous development, the specification’s steady-state role is a semantic regression harness: the analogue, at the state-machine level, of a test suite.

6 Verification Results and Their Limits

6.1 Results at the current baseline

Check	Layer	Scope	Status	Size
Structural proofs (4 graphs + composite)	L3	full	exhaustive	—
Coverage bijection (4 maps)	L2	full	exhaustive	83 entries
Solidity anchoring	L2	full	exhaustive, 0 failures	120 entries
Topology conformance	L4	consensus	exhaustive	42/42 arcs
TLC: concrete, 5 validators, no appeals	L5	all invariants	exhaustive	161,108 states
TLC: abstract, $N=50$, 5 appeals	L5	all invariants	exhaustive	2,849,563 states
TLC: concrete, 25 validators, 2 appeals	L5	all invariants	partial	297K states, no violations

State counts are distinct states explored. The partial run was terminated after throughput collapse; it found no violations but did not exhaust the space. Appeal-path coverage therefore rests on the abstract specification, which is exactly the role the abstraction was built for.

6.2 What these results do and do not certify

The exhaustive rows certify their properties *for the model sizes checked*: model checking quantifies over all behaviors of a finite instance, not over all instances. The abstraction argument (counts suffice for the checked properties) is a modeling judgment, made explicit in Section 4, not a theorem. The anchoring check certifies that the map references real code at the pinned revision; it does not certify that the map’s *semantic* annotations (preconditions, from-statuses) are correct—those were established by manual code analysis during the verification sessions and are the layer where human error survives. We consider labeling these boundaries part of the deliverable: a verification claim without its scope attached is how the February–June drift happened in the first place.

6.3 Known gaps

At the current baseline, three implementation mechanisms are documented but not yet modeled: the stuck-after-appeal restore guard (a reconciliation path for transactions wedged between queue and status), the administrative recovery subsystem, and the recomputation rewinds of subsequent transactions (scoped to the queue graph by a documented convention). The successor-round property S6 must also be updated from “fresh identities” to “fresh ballots with permissible identity reuse” and re-checked against the current baseline. The sets-to-counters abstraction phase is pending; until it lands, exhaustive concrete checking with active appeals is out of reach beyond small committees.

7 Related Work

TLA+ [3] and the TLC model checker [6] are the workhorses of specification-first verification in industry; Amazon’s experience report [4] established the pattern of using TLA+ on production distributed systems and is the closest methodological relative of this paper—our contribution relative to that line is the anchoring layer binding the specification corpus to a moving implementation, and the regression-harness framing backed by a concrete incident. Classical BFT verification targets protocol *designs* [5, 2]; verification of non-deterministic replicated execution [1] is adjacent but does not address semantic ambiguity, which in our verification stack is delegated to the companion statistical treatment rather than the model checker. The layered-artifact approach (graph as boundary object between paper, specification, and code) is, to our knowledge, not previously documented for a consensus protocol in production.

8 Conclusion

We presented a verification architecture whose design center is *re-verification*: a canonical graph as single source of truth, mechanical anchoring to a pinned implementation revision, and TLA+ specifications bound to the graph by an automatic topology check. Its value was demonstrated the unglamorous way—by four months of drift that the self-consistent layers missed and the anchored re-verification caught, surfacing a live race and three silently falsified protocol invariants in a production consensus implementation.

The findings feed back into the companion analyses: the voided-round states discovered here constrain Markov-chain models of the appeal ladder (the appeal-hang outcomes are now nondeterministic between two branches), and the fee-classification exposure of Finding 5.4 is an input to the companion analysis of appeal incentives. The forward agenda is: the sets-to-counters abstraction to reach realistic committee sizes; modeling of the recovery subsystem; completion of the governance specification track; and composition—checking that governance actions (rotation, freezes) preserve the consensus invariants mid-round, which is where the two state machines meet.

Institutional Disclosure

This work was conducted within GenLayerLabs Research. GenLayerLabs develops GenLayer, whose consensus implementation is the verification target of this paper. The layered architecture and the regression-harness methodology do not depend on that implementation.

References

- [1] Christian Cachin, Simon Schubert, and Marko Vukolić. Non-determinism in Byzantine fault-tolerant replication. In *20th International Conference on Principles of Distributed Systems (OPODIS 2016)*, volume 70 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 24:1–24:16. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2017.
- [2] Miguel Castro and Barbara Liskov. Practical Byzantine fault tolerance. In *Proceedings of the Third Symposium on Operating Systems Design and Implementation (OSDI)*, pages 173–186, New Orleans, LA, 1999. USENIX Association.
- [3] Leslie Lamport. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, Boston, 2002.
- [4] Chris Newcombe, Tim Rath, Fan Zhang, Bogdan Munteanu, Marc Brooker, and Michael Deardouff. How Amazon Web Services uses formal methods. *Communications of the ACM*, 58(4):66–73, 2015.
- [5] Marshall Pease, Robert Shostak, and Leslie Lamport. Reaching agreement in the presence of faults. *Journal of the ACM*, 27(2):228–234, 1980.
- [6] Yuan Yu, Panagiotis Manolios, and Leslie Lamport. Model checking TLA+ specifications. In *Correct Hardware Design and Verification Methods (CHARME)*, volume 1703 of *LNCS*, pages 54–66. Springer, 1999.

Appendices: Property Catalog and Artifacts

A Property Catalog

ID	Type	Statement (informal)
TypeOK	Safety	All variables within their domains.
CR1	Safety	Reveals never exceed commits (per round).
CR2	Safety	Committing/LeaderRevealing have the leader’s commit; Revealing likewise, <i>except</i> in a voided appeal round (Finding 5.2).
CR3	Safety	No validator reveal precedes the leader’s reveal (normal rounds).
S1	Safety	A finalized transaction has a result.
S2	Safety	At most one result per transaction.
S3	Safety	Leader-reveals-first in Revealing, <i>except</i> in a voided appeal round (Finding 5.3).
S4	Safety	Committee sizes follow the escalation schedule.
S5	Safety	Accepted + majority-agree implies quorum of agrees in live counters (guarded against restored states; Finding 5.4).
S6	Pending	Current target: successor rounds reset ballots; identity reuse is permitted. The pinned campaign checked an older fresh-identity formulation.
Tribunal	Safety	Tribunal resolution never mutates the transaction status directly.
L1–L3	Liveness	Every transaction reaches a terminal status; in-flight appeals and tribunals resolve; queues drain.

B Baselines and Artifacts

The verification corpus and the implementation revision are pinned per session. The reported machine-checking results correspond to consensus contracts at revision `40c4f458` (origin/main, 2026-06-29), with matching canonical-graph and TLA+ revisions. The protocol synopsis in Section 2 also records later-inspected mechanics needed by the companion papers; those mechanics are not retroactively covered by the older TLC run. In particular, the current S6 formulation is pending re-verification. The divergent-handler race (Finding 5.1) is tracked upstream as issue #1157 of the consensus repository; the falsified-invariant analysis is recorded in the specification repository’s findings document. All baseline checks described in Sections 3–6 run from a single CI entry point.